

Here is the implementation for **Stage 1: signup and login routes working**. Since your checkpoint example uses port 3000, these examples use **Express (JavaScript/TypeScript)** and **FastAPI (Python)**.

Node.js (Express & JavaScript)

javascript

```
import { createClient } from '@supabase/supabase-js';
import express from 'express';

const app = express();
app.use(express.json());

// Initialize Supabase client
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_ANON_KEY
);

// POST /auth/signup
app.post('/auth/signup', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: "Bad Request" });
  }

  const { data, error } = await supabase.auth.signUp({ email, password });

  if (error) {
    return res.status(400).json({ error: error.message });
  }

  return res.status(201).json(data);
});

// POST /auth/login
app.post('/auth/login', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: "Bad Request" });
  }
```

```

    }

    const { data, error } = await supabase.auth.signInWithPassword({ email, password });

    if (error) {
      return res.status(401).json({ error: "Invalid login credentials" });
    }

    return res.status(200).json({
      access_token: data.session?.access_token,
      refresh_token: data.session?.refresh_token
    });
  });
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

Use code with caution.

Python (FastAPI)

python

```

import os
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, EmailStr
from supabase import create_client, Client

app = FastAPI()

# Initialize Supabase client
supabase_url = os.environ.get("SUPABASE_URL")
supabase_key = os.environ.get("SUPABASE_ANON_KEY")
supabase: Client = create_client(supabase_url, supabase_key)

class AuthSchema(BaseModel):
    email: EmailStr
    password: str

@app.post("/auth/signup", status_code=status.HTTP_201_CREATED)
def signup(credentials: AuthSchema):
    # Pydantic handles missing/empty fields validation natively and throws 422/400

```

```
try:
    response = supabase.auth.sign_up({
        "email": credentials.email,
        "password": credentials.password
    })
    return response
except Exception as e:
    raise HTTPException(status_code=400, detail=str(e))

@app.post("/auth/login", status_code=status.HTTP_200_OK)
def login(credentials: AuthSchema):
    try:
        response = supabase.auth.sign_in_with_password({
            "email": credentials.email,
            "password": credentials.password
        })

        return {
            "access_token": response.session.access_token,
            "refresh_token": response.session.refresh_token
        }
    except Exception:
        # Catching Supabase auth execution failures (wrong email/password)
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid login credentials"
        )
```